WorkFlow Lojistik AI Operasyon

LM Studio GGUF model entegrasyonu, PostgreSQL veri akışı ve React/FastAPI operasyon arayüzü proje raporu

Rapor kapsamı: projenin amacı, başlangıçtan bugüne yapılan geliştirmeler, model entegrasyonu, veritabanı tasarımı, otomatik mesaj simülasyonu, manuel kontrol akışı, test edilen noktalar ve teslim notları.

Başlık	Detay
Frontend	React + Vite, sabit port 5173
Backend	FastAPI, port 8001
Model	LM Studio Local Server, OpenAI uyumlu API, port 1234
Database	PostgreSQL, lojistik_workflow
Veri kaynağı	mesajlar.json üzerinden 5 dakikada 2 mesaj import

1. Yönetici Özeti

WorkFlow projesi, lojistik WhatsApp mesajlarını ham veri olarak alıp yapay zeka destekli sipariş kayıtlarına dönüştüren bir operasyon uygulamasıdır. Sistem; lokal çalışan LM Studio modeli, FastAPI backend, PostgreSQL veritabanı ve React arayüzünden oluşur.

Proje başlangıcında modelin GGUF formatına hazırlanması ve LM Studio üzerinden çalıştırılması hedeflendi. Devamında backend modeli OpenAI uyumlu LM Studio API ile çağıracak şekilde uyarlandı, AI sonuçları PostgreSQL'e yazıldı, arayüzde ham mesaj, çözümleme, manuel kontrol, sipariş ve workflow ekranları birbirine bağlandı.

Son durumda proje; 5173 frontend portu, 8001 backend portu ve 1234 LM Studio portu ile çalışacak şekilde düzenlenmiştir. Dashboard, manuel kontrol ve AI hata ayrımı gerçek API verisine göre güncellenmiştir.

2. Başlangıçtan Bugüne Yapılanlar

• PostgreSQL tablo yapısı netleştirildi: gönderenler, ham mesajlar, çözümleme kayıtları, siparişler, durum geçmişi, doğrulama sorunları ve JSON import cursor tablosu oluşturuldu.

• LM Studio entegrasyonu yapıldı: GGUF model lokal server üzerinden backend'e bağlandı ve fake model yerine LMStudioModelClient aktif hale getirildi.

• AI çözümleme akışı tamamlandı: model cevabı normalize edildi, validator ile eksik veya standart dışı alanlar manuel kontrole yönlendirildi.

• Ham Mesajlar ve AI Çözümleme ekranları tutarlı hale getirildi: son çözümleme kaydı gösteriliyor, manuel kontrol ve hata durumları ayrıştırılıyor.

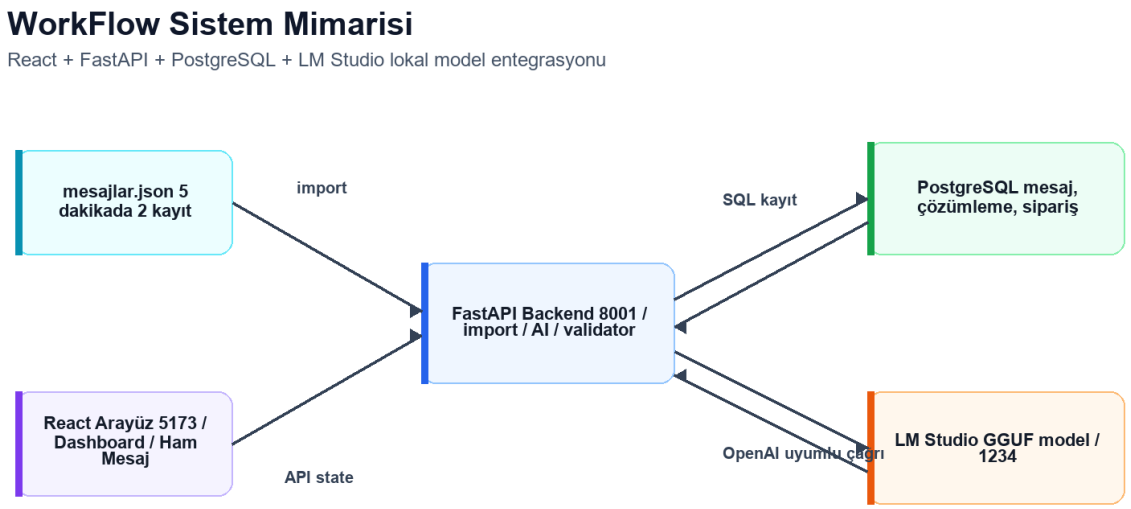
• Manuel kontrol sonrası kaydetme akışı düzeltildi: eksikler tamamlanınca mesaj durumu 'islendi', sipariş durumu 'inceleniyor' oluyor ve açık sorunlar kapatılıyor.

• mesajlar.json simülasyonu eklendi: backend açılınca 5 dakikada bir sıradaki 2 mesaj içeri alınır ve AI çözümlemeye gönderilir.

• Dashboard güncellendi: toplam mesaj, işlenen, manuel kontrol, hata, sipariş ve açık sorun sayıları gerçek backend summary verisinden okunur.

• Teslim hazırlığı yapıldı: README güncellendi, PostgreSQL ve LM Studio kurulum bilgileri eklendi, gereksiz çalışma/cache dosyaları temizlendi.

3. Sistem Mimarisi



Mimari; veri kaynağından gelen mesajların backend tarafından alınması, PostgreSQL'e yazılması, LM Studio modeline gönderilmesi ve React arayüzünde operasyonel duruma dönüştürülmesi üzerine kuruludur.

4. AI Çözümleme ve Durum Akışı



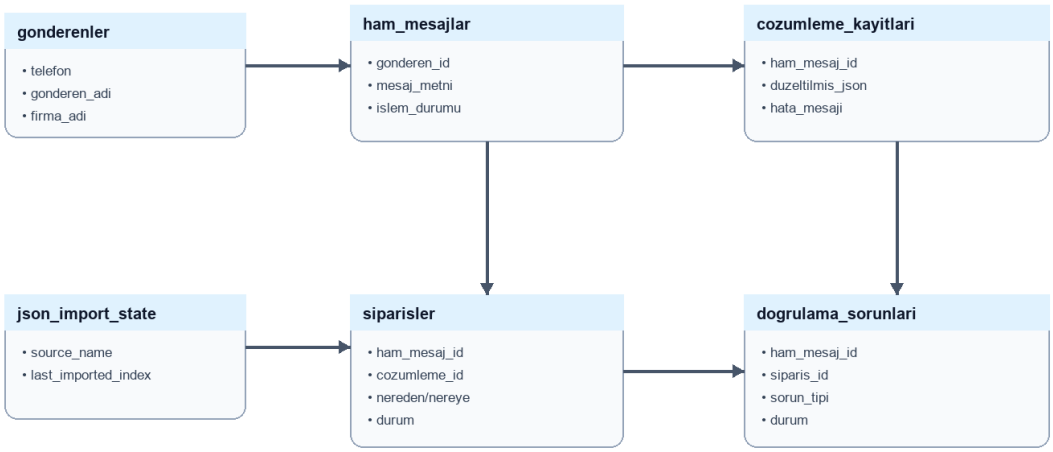
AI sonucunda üç ana durum oluşur: 'islendi' sorunsuz kayıtları, 'manuel_kontrol' eksik/emin olunmayan alanları, 'hata' ise model bağlantısı veya JSON parse sorunlarını temsil eder.

Durum	Anlam	Arayüz Aksiyonu
islendi	AI sonucu kullanılabilir ve sipariş oluşmuştur.	Sonucu Gör

manuel_kontrol	AI sonucu var ancak bazı alanlar eksik veya emin değil.	Manuel Kontrol
hata	Model bağlantısı, timeout veya JSON parse hatası vardır.	Hatalı / Tekrar Dene
isleniyor	Backend modeli çağırıyor veya import akışı çalışıyor.	Bekleme
bekliyor	Mesaj henüz AI çözümlemeye girmedi.	AI Çözümle

5. PostgreSQL Veritabanı Yapısı

PostgreSQL Tablo İlişkileri



Veritabanı yapısı operasyonel izlenebilirlik için ayrıştırılmıştır. Ham mesaj, model çözümlemesi, oluşan sipariş, doğrulama sorunu ve sipariş durum geçmişi ayrı tablolarda tutulur.

Tablo	Görev
gonderenler	Mesaj atan kişi/firma ve telefon bilgisini tutar.
ham_mesajlar	WhatsApp/json kaynaklı orijinal mesajı ve işleme durumunu tutar.
cozumleme_kayitlari	Model cevabı, düzeltilmiş JSON ve hata bilgisini saklar.
siparisler	AI veya manuel kontrol sonrası oluşan operasyon kayıtlarıdır.
dogrulama_sorunlari	Eksik alan, standart dışı değer ve model hatalarını izler.
json_import_state	mesajlar.json dosyasında kaldığı import indexini tutar.

6. LM Studio ve Model Entegrasyonu

Model tarafında hedef, Qwen tabanlı lojistik modeli GGUF formatında LM Studio'ya yükleyip backend içinden normal bir API modeli gibi kullanmaktı. Bu amaçla LM Studio'nun OpenAI uyumlu /v1/chat/completions endpointi kullanıldı.

Konu	Son Durum
Model çalışma şekli	LM Studio Local Server üzerinden lokal GGUF model çağrısı
Backend client	LMStudioModelClient
Server URL	http://127.0.0.1:1234
Timeout	180 saniye

Max tokens	1600; uzun mesajlarda yarım JSON riskini azaltmak için yükseltildi
Fallback	Fake fallback kapalı

Gözlenen önemli iyileştirme: İlk denemelerde uzun mesajlarda düşük token sınırı nedeniyle JSON cevabının yarım kalma riski vardı. Bu nedenle LM Studio max token üst sınırı 400'den 1600'e çıkarıldı ve parse hatasında ham model cevabının saklanması sağlandı.

7. Arayüz Özellikleri

• Dashboard: sistem/model durumu ve güncel operasyon sayıları.

• Ham Mesajlar: gelen mesaj listesi, AI durumu ve aksiyon butonları.

• AI Çözümleme: ham mesaj, model cevabı, düzeltilmiş JSON ve oluşan sipariş kartları.

• Manuel Kontrol: eksik alanları düzenleme, sipariş sayısını değiştirme ve kaydetme.

• Siparişler: oluşan siparişleri listeleme ve karar durumunu değiştirme.

• İş Sırası: operasyon durumlarını kanban benzeri akışta izleme.

8. Test ve Doğrulama Özeti

Test	Beklenen Sonuç
GET /api/health	Database connected cevabı döner.
GET /api/model/status	LMStudioModelClient ve LM Studio URL bilgisi görünür.
POST /api/import/tick	Sıradaki 2 mesaj import edilir ve AI çözümleme başlar.
Manuel kontrol kaydı	Eksikler tamamlanırsa mesaj 'islendi' olur ve açık sorunlar kapanır.
Frontend 5173	Vite 5173'e sabitlenmiştir; 5174'e kaçmaz.

9. Teslim ve Kurulum Notları

• README.md içinde LM Studio, PostgreSQL, backend, frontend ve kontrol komutları anlatılmıştır.

• Tablo kurulum dosyası backend/database_schema.sql içindedir.

• Örnek verili SQL dosyası backend/lojistik_workflow_schema_ve_ornek_veri.sql içindedir.

• Teslimde node_modules, dist, backend/.venv ve __pycache__ klasörleri gönderilmek zorunda değildir.

• backend/.env dosyası kalacaksa alan kişi DATABASE_URL içindeki PostgreSQL şifresini kendi ortamına göre kontrol etmelidir.

10. Sonuç

WorkFlow projesi, lokal çalışan GGUF modelin operasyonel bir lojistik uygulamasına entegre edildiği uçtan uca bir prototipten teslim edilebilir bir v1 yapısına taşınmıştır. Sistem; ham mesaj alma, AI çözümleme, doğrulama, manuel düzeltme, sipariş üretme ve operasyon akışına alma adımlarını tek akışta birleştirir.